

Web-programozás 1 – Előadás Beadandó feladat

Édes Liget cukrászda webalkalmazás dokumentációja

Készítette:

Harmatiné Pásztor Éva (JBDAQO)

Deák Dávid (OPFHRV)

Projekt elérési útvonala és belépéshez szükséges adatok:

- GitHub projekt URL cím: <https://github.com/harmatinepasztoreva/webprog-beadando>
- Weboldal URL cím: <http://edesliget.nhely.hu/>
- Internetes tárhely és adatbázis belépéséhez szükséges FTP és URL cím:
 - Internetes tárhely: <https://www.nethely.hu/>
 - Email: info.alfadelta@gmail.com
 - Jelszó: eviesdavidwebprog18
 - FTP kiszolgáló: <ftp.nethely.hu>
 - Felhasználó: eviesdavid
 - Jelszó: eviesdavidwebprog18

Tartalomjegyzék

Bevezetés	3
1. A projekt előkészítése	3
2. A közös arculat kialakítása	4
3. Index.html	6
4. Javascript.html	7
a) CRUD alkalmazás kialakítása	7
b) A JavaScript oldal formázása és az egységes megjelenés biztosítása	9
5. React.html	10
6. A reszponzív megjelenés átdolgozása	11
7. Spa.html	11
a) Első változat kialakítása	11
b) Véglegesítés	12
8. Fetchapi.html	13
9. Axios.html	14
10. Oojs.html	15
11. A Nettarhely.hu-ra való feltöltés	17
12. Rövid használati útmutató a honlaphoz	18
13. Összegzés	19
14. Források	20

Bevezetés

Jelen dokumentáció célja annak részletes bemutatása, hogyan készült el a Webprogramozás I. előadás tantárgy beadandó feladata keretében létrehozott webalkalmazásunk. A feladat lényege egy több menüpontból álló, különböző webes technológiákat felhasználó alkalmazás elkészítése volt, amelyben a hagyományos HTML–CSS alapokra építve JavaScriptes, Reactes, Fetch API-s és Axiosos megoldásokat is alkalmaznunk kellett. A projektet kétfős csoportban valósítottuk meg, ezért már a munka kezdetén fontos szempont volt a feladatok összehangolása, a közös munka megszervezése, valamint a fejlesztési folyamat átlátható dokumentálása. A közös fejlesztéshez a GitHubot és a GitHub Desktop alkalmazást használtuk, amely jelentősen megkönnyítette a verziókövetést és a párhuzamos munkavégzést. Mivel a beadandóhoz adatbázist is kellett választani, a fejlesztés megkezdése előtt áttekintettük a megadott adatbázis-forrásokat, és végül a „1-14-Cukrászda-3 táblás” elnevezésű cukrászda adatbázis mellett döntöttünk. Ezt az adatbázist azért választottuk, mert olyan témát szerettünk volna, amely a tartalma miatt könnyen érthető, esztétikusan megjeleníthető, és jól illeszkedik egy látványos webes felülethez, illetve könnyen található hozzá a webes felületeken képanyag. A projekt során végig törekedtünk arra, hogy az alkalmazás ne pusztán a kiadott feladatok technikai megvalósítása legyen, amelyet a megadott "Web-programozás-1-Előadás" pdf-ben bemutatott technikai megoldásokat valósítsuk meg, hanem vizuálisan is egységes, átgondolt és felhasználóbarát rendszert alkossunk. Ezen okok miatt a fejlesztés során nemcsak a működésre, hanem az arculatra, a színekre, a logóra, a menü megjelenésére és a kisebb képernyőkön való használhatóságra is nagy hangsúlyt fektettünk és sok időt töltöttünk annak megvalósításával. A dokumentációban fejezetenként mutatjuk be az egyes oldalak kialakítását, megvalósításának lépéseit és a munkafolyamat közben felmerült problémákat, illetve azok megoldását. Célunk az volt, hogy a leírásból világosan követhető legyen, pontosan hogyan és hol teljesítettük a beadandó feladat egyes követelményeit.

1. A projekt előkészítése

A fejlesztési folyamat első szakasza nem közvetlenül a kódolással kezdődött, hanem a közös munka technikai feltételeinek megteremtésével. Mivel a csapat egyik tagja már korábban is használt GitHubot, ezért neki már rendelkezésére állt saját felhasználói fiók, így csak a másik tagnak kellett a regisztrációt elvégeznie. Ezután létrehoztuk a projekt számára a „webprog-

beadando” nevű nyilvános repository-t, amely a teljes fejlesztési folyamat központi helyszíne lett.

Már a kezdetektől fontosnak tartottuk, hogy a projekt ne egyetlen nagy feltöltésből álljon, hanem több, egymást követő állapotban kerüljön fel a GitHubra, mivel a feladat értékelésének része volt a verziókövető rendszer tudatos használata is. Mindketten telepítettük a GitHub Desktop alkalmazást, mert ez nagyban leegyszerűsítette a közös munkát, a commitok kezelését, a változások áttekintését és az egyes részfeladatok elkülönítését.

A fejlesztés megszervezésének másik fontos eleme az adatbázis kiválasztása volt. A feladatléírásban megadott online mappában többféle adatbázis-forrás állt rendelkezésre, ezért a HTML oldalak létrehozása előtt ezeket részletesen átnéztük. Végül a „1-14-Cukrászda-3 táblás” elnevezésű adatbázis mellett döntöttünk, mert úgy éreztük, ez a téma mindkettőnk számára könnyen átlátható, jól értelmezhető és vizuálisan is jól feldolgozható. Egy műszaki vagy erősen szakmai adatbázis esetén jóval több idő ment volna el pusztán a tartalom értelmezésére, míg a süteményekhez kapcsolódó adatok mindenki számára ismerősek voltak. Emellett az is a cukrászdás téma mellett szólt, hogy az interneten könnyen lehet hozzá illő képeket, inspirációkat, színvilágot és arculati elemeket találni.

2. A közös arculat kialakítása

A weboldal fejlesztésének gyakorlati megvalósítását nem rögtön az egyes oldalak tartalmi kidolgozásával kezdtük, hanem először létrehoztuk a feladatban meghatározott összes HTML oldalt:

- index.html
- javascript.html
- react.html
- spa.html
- fetchapi.html
- axios.html
- oojs.html.

Ezeket ebben a korai szakaszban még nem töltöttük fel érdemi tartalommal, mert ekkor elsődlegesen az volt a célunk, hogy a menüsor kialakításához már rendelkezésre álljanak a

megfelelő hivatkozási pontok. Úgy gondoltuk, hogy hosszú távon sokkal átláthatóbb és tudatosabb fejlesztési folyamatot eredményez, ha már a menü sor kialakításánál megvannak azok az oldalak, amelyekre később a tényleges tartalom kerülni fog. Ez a megközelítés azért is bizonyult hasznosnak, mert már a kezdetektől láthatóvá vált a teljes alkalmazás felépítése, így nem elszigetelt oldalakat fejlesztettünk, hanem egy összefüggő weboldalrendszert.

Miután a szükséges HTML fájlok létrejöttek, az index oldalon először csak az alábbi legfontosabb kötelező elemeket alakítottuk ki.:

- Fejlécbe ki lett írva H1-es címsorral: Web programozás-1 Előadás Házi feladat
- Láblécbe beírtuk a készítők nevét és Neptun kódját:

<footer>

<p>Készítette: Harmatiné Pásztor Éva (JBDAQO) - Deák Dávid (OPFHRV)</p>

Ez még nem a főoldal végleges kidolgozását jelentette, hanem inkább egy stabil váz létrehozását, amelyre később vissza tudtunk térni. Ezzel párhuzamosan kezdtük el a közös vizuális arculat kialakítását is.

Kezdetben egy **index.css** nevű fájlt hoztunk létre, de rövid időn belül világossá vált, hogy ez a stíluslap nem kizárólag a főoldalhoz fog tartozni, hanem az egész alkalmazás közös megjelenését fogja meghatározni, így a fájl nevét **menu.css**-re módosítottuk. Ebben a stíluslapban alakítottuk ki az oldal alapvető színvilágát, a menüsor szerkezetét, a fejléc és a lábléc közös megjelenését, valamint minden olyan vizuális elemet, amelyet később az összes oldal felhasznált.

A színek kiválasztásánál tudatosan olyan árnyalatokat kerestünk, amelyek egy valóban létező cukrászda hangulatát idézik. Hosszas keresés után végül két meghatározó árnyalat mellett döntöttünk: a **#6b3e26** kódú sötétbarna és a **#f8e8d0** kódú világos bézs szín került a központba. Ezek a színek jól illenek a cukrászda tematikához, ugyanakkor elegendő kontrasztot is biztosítanak ahhoz, hogy a feliratok és a különböző felületi elemek jól olvashatóak maradjanak. A színek kiválasztásához a <https://colors.co/> weboldalt használtuk, mert ez nagy segítséget nyújtott abban, hogy egymással harmonizáló színpárt találjunk. Ebben a szakaszban tehát még nem a főoldal végleges szöveges feltöltése volt a középpontban, hanem egy olyan egységes vizuális alap létrehozása, amelyre a későbbi oldalak következetesen ráépülhettek.

3. Index.html

Az **index.html** oldal kialakítása több szakaszban történt, és a végleges főoldal nem egyszerre készült el. Ahogy korábban említettem, a fejlesztés elején ezen az oldalon először csak a legszükségesebb szerkezeti alapokat hoztuk létre. Ide került a beadandó által kötelezően előírt H1-es főcím, vagyis a „Web programozás-1 Előadás Házi feladat” felirat, valamint a lábléc is. Ez a korai változat még nem tekinthető a főoldal végleges tartalmi megoldásának, inkább egy alapváz volt, amely biztosította, hogy a későbbi fejlesztések során legyen egy stabil szerkezeti kiindulópontunk. A fejlesztési folyamat következő szakaszában nem rögtön a főoldal részletes kidolgozását folytattuk, hanem a **javascript.html** megvalósítására tértünk át. Ennek az volt az oka, hogy a beadandó egyik legfontosabb funkcionális része a JavaScript alapú CRUD alkalmazás volt, és ezt szerettük volna minél előbb működőképes állapotba hozni.

Miután a **javascript.html** már megfelelően működött, visszatértünk a főoldalhoz, és ekkor kezdtük el annak tényleges vizuális és tartalmi kiteljesítését. Ebben a szakaszban készült el a cukrászda logója is, amely a weboldal arculatának egyik meghatározó elemévé vált. A logó megtervezéséhez a <https://www.canva.com/> weboldal segítségét vettük igénybe. A cukrászdanak közösen az „Édes Liget” nevet választottuk, amely jól hangzó és a témához illő elnevezés. Külön figyeltünk arra is, hogy a név ne legyen túlságosan általános, és ne találjunk pontosan ilyen nevű cukrászdát Kecskemét környékén. A logó kialakításánál egy süteményhez kapcsolódó motívumot választottunk, és ez köré helyeztük el az „Édes Liget” cukrászda nevét. Az elkészült logót a fejléc bal felső részére helyeztük el, közvetlenül a H1-es főcím mellé. Mivel a weboldal minden HTML oldala a **menu.css** stíluslapra hivatkozik, a logó egységesen minden oldalon megjelent, és ez nagyban hozzájárult ahhoz, hogy az alkalmazás összefüggő, tudatosan felépített arculatot kapjon.

A háttér kialakítása szintén csak ebben a későbbi szakaszban történt meg, vagyis nem a legelső főoldali munkák része volt. Kezdetben többféle háttérképes megoldással próbálkoztunk: tortákat és süteményeket ábrázoló képeket szerettünk volna az oldal mögé helyezni, mert ezek tematikusan jól illettek volna a cukrászdas tartalomhoz. A gyakorlat azonban azt mutatta, hogy ezek a képek különböző képernyőméreteken nem viselkedtek megfelelően. Elcsúsztak, arányaik megváltoztak, és gyakran rontották a szövegek olvashatóságát is. Emiatt végül egy letisztultabb és használhatóbb megoldást választottunk: magát a logót helyeztük el vízjelszerű háttérelemként az oldal középső részén. Ezt kisebb

képernyőre váltáskor elrejtettük, így a háttér nem zavarta az olvashatóságot, mégis adott egyedi karaktert az oldalnak.



1. ábra Logó elhelyezése



2. ábra Logó nélküli háttér kis képernyős elrendezésnél

Ezt követően kezdtük el az index oldal tényleges tartalmi feltöltését. A főoldalt négy különálló, jól áttekinthető tartalmi egységre osztottuk, amelyek H2-es címsorral jelentek meg. Ezek a :

- „Történetünk”,
- „Történeink története”,
- „Elhelyezkedésünk” és
- „Rendelés leadás módja”

című részek lettek. Ezeket a blokkokat közösen találtuk ki, és más cukrászdák honlapjait is áttekintve töltöttük fel tartalommal.

Ebben a szakaszban valósítottuk meg a reszponzív kinézetet is, vagyis azt a működést, hogy kisebb nézetre váltva a hagyományos menü helyett egy bal oldalon megjelenő hamburger menü legyen elérhető. Ez nemcsak technikai követelményként volt fontos, hanem használhatósági szempontból is, mivel így a főoldal mobilos nézetben is átlátható maradt. A főoldal végül tehát nem a fejlesztés legelején készült el teljes formájában, hanem több egymásra épülő munkafázis eredményeként vált a weboldal központi, bemutatkozó felületévé.

4. Javascript.html

a) CRUD alkalmazás kialakítása

A főoldal kezdetleges szintű elkészülte után a javascript.html oldal fejlesztésével kezdtünk foglalkozni. Ennél az oldalnál a beadandó előírása szerint egy JavaScript alapú CRUD alkalmazást kellett elkészítenünk, amely a választott adatbázis egyik fájljának adatait dolgozza fel, és az adatokat tömbben tárolja. Az alapot a tananyagban "Web-programozás-1-Előadás" pdf-ben található "CRUD alkalmazás – tömbbel - Oldal frissítésekor újraindul az alkalmazás => AJAX-Fetch API ezt majd megoldja" címszó alatt található JavaScript CRUD example részt vettük alapul és ezen anyagot használtuk fel.

JavaScript CRUD example

Full Name*

Email Id

Salary

City

Submit

Full Name	Email Id	Salary	City	Actions
John Smith	data1@gmail.com	2000	London	Edit Delete
Tom Brown	data2@gmail.com	2500	Paris	Edit Delete

3. ábra: Javascript example

Ezt a mintát szó szerint kiindulási pontként használtuk, majd a saját témánkhoz igazítottuk. A táblázat szerkezetét alapvetően meghagytuk, mert a kiinduló példa jól átlátható és működőképes volt, viszont a mezőneveket teljes mértékben átírtuk a cukrászdai tartalomnak megfelelően, az alábbiak szerint:

- Full Name → Id*
- Email ID → Sütemény neve
- Salary → Sütemény típusa
- City → Díjazott
- Submit → Mentés

Az eredeti employee-form elnevezést szintén lecseréltük, és a saját alkalmazásunkban suti-form néven használtuk tovább, mert így jobban kifejezte a tartalmat. Ugyanez történt az alsó adats megjelenítő résszel is: az employees-table helyett suti-table elnevezést alkalmaztunk. Az „Edit” és „Delete” gombok szintén magyar feliratot kaptak, vagyis „Módosítás” és „Törlés” formában jelentek meg. A „Sütemény típusa” mezőnél már nem egyszerű szabad szövegbevitelt alkalmaztunk, hanem előre összeállított választható listát készítettünk. Ennek előkészítéséhez

az adatbázist előbb Excel táblázatba rendeztük, majd a típusokat összegyűjtöttük és option value formában beépítettük a felületbe. Így a felhasználó csak a meghatározott típusok közül választhatott, például vegyes, sós teasütemény, bejgli, torta, tortaszelet, pite, tejszínes sütemény, édes teasütemény, különleges torta vagy krémes. Hasonló megoldást alkalmaztunk a „Díjazott” mezőnél is, ahol „igen” és „nem” opciót lehetett kiválasztani. Miután az átnevezések és a mezők testreszabása elkészült, alaposan teszteltük az oldal működését, és csak akkor kezdtük el a megjelenés finomhangolását, amikor már a CRUD logika megfelelően működött.

b) A JavaScript oldal formázása és az egységes megjelenés biztosítása

Miután a javascript.html működése megfelelővé vált, a következő fontos szempont az volt, hogy az oldal kinézete is illeszkedjen a főoldalon kialakított arculathoz. Emiatt elhelyeztük a javascript.html fájlban a menu.css hivatkozást, így ez az oldal is ugyanazt a menüszerkezetet, színvilágot és általános stílust örökölte, mint az index.html. Emellett külön javascript.css fájlt is készítettünk, mert a CRUD táblázat és az űrlap már speciálisabb formázást igényelt. Ebben a külön stíluslapban ugyanazt a két meghatározó színt használtuk, mint a közös arculat kialakításánál: a sötétbarna és a világos bézs árnyalatot. Ennek köszönhetően a JavaScript oldal nem tűnt különálló elemnek, hanem természetesen illeszkedett az egész weboldal megjelenéséhez. A mezők és a gombok elrendezését ezért úgy alakítottuk ki, hogy az oldal egyértelműen használható maradjon, és a felhasználó számára azonnal látszódjon, hol lehet új elemet felvenni, illetve melyik sor módosítható vagy törölhető.

The screenshot shows a web application interface for 'Web programozás-1 Előadás Házi feladat'. It features a navigation bar with links to 'Főoldal', 'JavaScript', 'React', 'SPA', 'Fetch API', 'Axios', and 'OOJS'. The main content area contains a form for adding a new item with fields for 'Id*', 'Sütemény neve', 'Sütemény típusa' (a dropdown menu), and 'Díjazott' (a dropdown menu). A 'Mentés' button is located at the bottom right of the form. Below the form is a table with the following data:

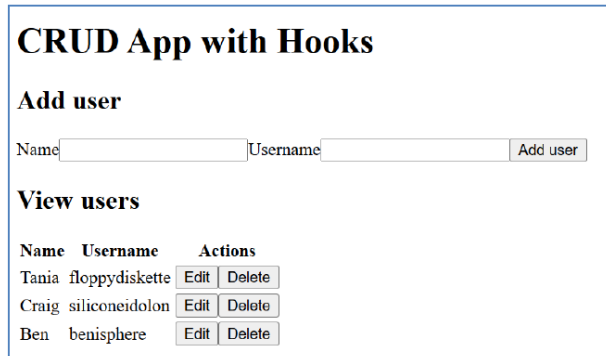
Id	Sütemény neve	Sütemény típusa	Díjazott	Műveletek
1	Süni	vegyes	nem	Módosítás Törölés

At the bottom of the page, it says 'Készítette: Harmatiné Pásztor Éva (JBDAQO) - Deák Dávid (OPFHRV)'.

4. ábra javascript.html

5. React.html

Miután a főoldal és a JavaScript alapú CRUD alkalmazás megfelelően működött, a következő nagy feladatrészt a react.html menüpont megvalósítása volt. Ennél az oldalnál a beadandó előírása szerint React segítségével kellett CRUD alkalmazást készítenünk, amely szintén a választott adatbázis egyik fájljának adatait használja fel. A fejlesztéshez a tananyagban szereplő React-CRUD megoldást, valamint a tanár úr által biztosított megoldások.zip fájlt használtuk alapként.



CRUD App with Hooks

Add user

Name Username

View users

Name	Username	Actions
Tania	floppydiskette	Edit Delete
Craig	siliconeidolon	Edit Delete
Ben	benisphere	Edit Delete

5. ábra React CRUD minta

Az első lépés az volt, hogy a szükséges fájlokat átemeltük a saját projektünkbe. Ezek közé tartozott a src/App.jsx, a src/main.jsx, továbbá az AddUserForm, EditUserForm és UserTable komponensek. Ezzel létrejött a React alkalmazás működési alapja, amelyet ezt követően a saját témánknak megfelelően kellett átalakítani.

A react.html formázásánál fontos szerepet kapott az egységes megjelenés, ezért a head részbe bekerült a menu.css-re mutató hivatkozás, a body részhez pedig átmásoltuk az index oldal topbar szerkezetét. Emellett a navigációban az aktív menüpontot is átállítottuk úgy, hogy a React oldalnál legyen kiemelve. A tartalom elrendezéséhez a <main class="content"> részt is beillesztettük, így a React oldal vizuálisan is ugyanabba a rendszerbe került, mint a főoldal. A komponensek elnevezését teljes mértékben a saját témánkhoz igazítottuk. A UserTable helyére SutemenyTable, az AddUserForm helyére AddSutemenyForm, az EditUserForm helyére pedig EditSutemenyForm került. Ez a névváltás nem pusztán formai kérdés volt, hanem azt szolgálta, hogy a teljes alkalmazás nyelvezete következetes maradjon. A táblázatokban elvégeztük a szükséges felíratmódosításokat is, hogy azok illeszkedjenek a javascript.html oldalon kialakított suti-form és suti-table logikájához. A minta React táblázathoz képest plusz oszlopokat is létrehoztunk, hogy a tartalom összhangban legyen a JavaScript változattal. Ezt követően az App.css fájlt saját igényeink szerint átalakítottuk, majd később átneveztük React.css névre. Ebben a stíluslapban a JavaScript oldalhoz igazítottuk az elrendezést, vagyis ugyanazt a színvilágot és nagyon hasonló szerkezeti megoldásokat alkalmaztuk. A React oldal fejlesztése során tehát nem csupán az volt a cél, hogy működőképes CRUD alkalmazás

készüljön, hanem az is, hogy a felhasználó számára természetesnek hasson az átmenet a JavaScript és a React alapú felület között.

6. ábra react.html

6. A reszponzív megjelenés átdolgozása

A react.html tesztelése során észleltük, hogy a korábban kialakított reszponzív megjelenés ezen az oldalon már nem működik megfelelően. Míg a főoldal és a JavaScript oldal esetében a kisebb képernyőméretre váltás viszonylag jól kezelhető volt, a React oldalon az elrendezés elcsúszott, a táblázat nem mutatott megfelelően, és a bal oldalon elhelyezett hamburger menü sem illeszkedett harmonikusan az új szerkezethez. A probléma megoldása érdekében átdolgoztuk az oldal elrendezését, és a hamburger menüt áthelyeztük a menüsor jobb oldalára. Ezzel az apró, de lényeges módosítással az oldal sokkal rendezettebbé vált kisebb képernyőn is. A korábbi beállításokat így olyan módon finomítottuk, hogy a menü jól látható maradjon, ugyanakkor ne törje meg a React táblázat és az űrlap vizuális egységét. Ez a módosítás jól mutatja, hogy a fejlesztés során többször is szükség volt visszalépésre és újra tervezésre.

7. Spa.html

a) Első változat kialakítása

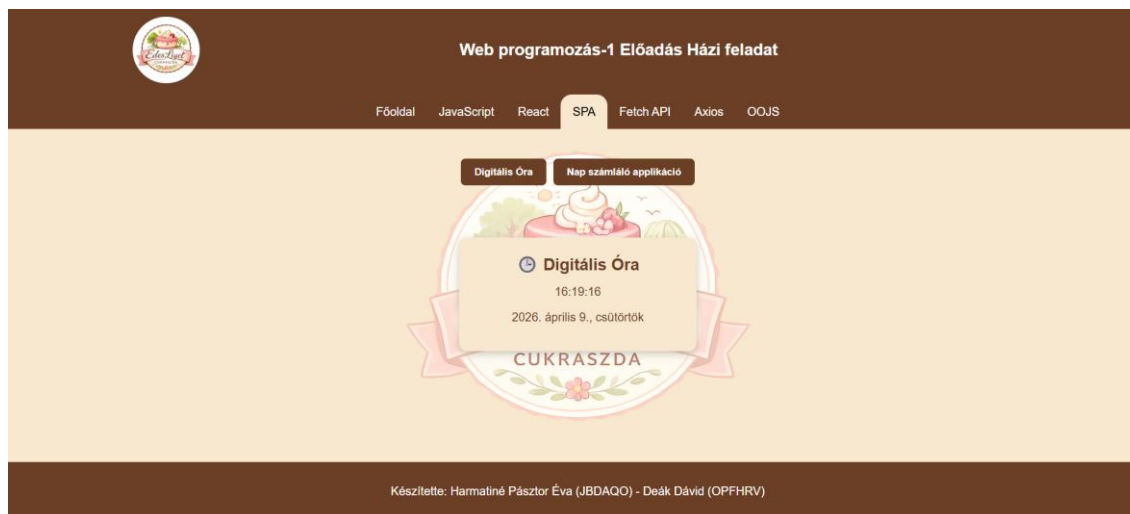
A React CRUD elkészítése után a SPA, vagyis az egyoldalas alkalmazás megvalósítása következett. Ennél a feladatrésznél a beadandó külön kiemelte, hogy két menüponttal

rendelkező Single Page Applicationt kell készíteni, ahol a két menüponthoz két külön React alkalmazás tartozik. A fejlesztés kezdeti szakaszában nem rögtön saját kiválasztott alkalmazásokat kerestünk, hanem először a működés logikáját akartuk stabilan megvalósítani. Ezért a megoldasok.zip fájlból átmásoltuk a tic-tac-toe és a calculator alkalmazást. Ezek jó alapot adtak ahhoz, hogy a menüváltás logikáját kipróbáljuk: a spa.html oldalon két gomb jelent meg, és ezekre kattintva ugyanazon az oldalon belül más-más alkalmazás töltődött be. Ez a működés jól szemléltette a SPA lényegét, hiszen az oldal nem töltődött újra, csak a megjelenített tartalom változott meg. A kezdeti cél tehát az volt, hogy a mechanizmus biztosan jól működjön. Ezt követően kezdtük el az oldal megjelenésének hozzáigazítását a már kialakított közös arculathoz. Az index oldalról átmásoltuk a topbar és a footer szerkezetét, illetve itt is elhelyeztük a menu.css-re mutató hivatkozást.

b) Véglegesítés

Miután a SPA működésének alapelve már stabilan megvolt, megkezdtük a két végleges React alkalmazás kiválasztását. Ehhez a tananyagban szereplő linket használtuk, vagyis az <https://github.com/ianshulx/React-projects-for-beginners> GitHub gyűjteményt. Több projektet is megvizsgáltunk, például a weight convertert, a dice-appot, a memory-card-game-et és más hasonló alkalmazásokat. A választás során egyszerre kellett figyelembe venni a feladat nehézségi követelményeit és a projekt méretét is, mivel a feltöltésnél az is szempont volt, hogy a GitHubon kezelhető maradjon a mappa mérete. Végül két egyszerűbb, de jól használható és látványos alkalmazás mellett döntöttünk: a digital clock és a date counter app került be a végleges SPA oldalra. Ezek nehézségi szintje megfelelően illeszkedett a calculator és tic-tac-toe példákhoz. A kiválasztott alkalmazások App.jsx kódját megkerestük, majd ezeket a korábban használt példák helyére másoltuk be. Ennek során több módosítást is el kellett végezni. Például a forrásalkalmazások saját CSS hivatkozásait töröltük, mert azt szerettük volna, hogy a megjelenés a saját, már kialakított stílusunkhoz igazodjon. Az SpaPage.jsx fájlban átírtuk a hivatkozásokat is, így a menügombok már a digital clock és a date counter alkalmazásokat töltötték be. Miután a működés ezzel az új tartalommal is stabillá vált, megkezdtük a végső formázást. A menüválasztó gombokat a saját színpalettánkhoz igazítottuk, középre rendeztük, és ügyeltünk arra, hogy az oldal egészének hangulatát erősítsék. Emellett a

két alkalmazás magyarosítása is megtörtént, vagyis a feliratokat magyar nyelvre módosítottuk, illetve a napokat is magyarul jelenítettük meg.

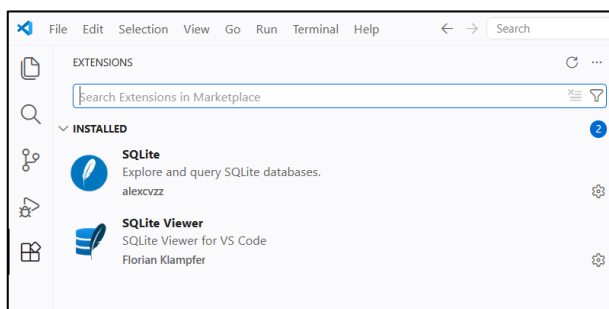


7. ábra spa.html

A fejlesztés lezárását követően utólag ellenőriztük a beadandó követelményeit, és ekkor vettük észre, hogy a feladat előírja a React alkalmazások „src” és „dist” mappáinak feltöltését a GitHub repozitóriumba. A kezdeti feltöltés során a kért mappa még nem szerepelt a repozitóriumban, ezért ezt utólag pótoltuk. A React alkalmazásokat Create React App környezetben készítettük, amely nem „dist”, hanem „build” nevű mappát hoz létre a „npm run build” parancs futtatásakor. Ennek megfelelően a két alkalmazás (digital_clock és datecounter) esetében a „build” mappák kerültek feltöltésre, amelyek a „dist” mappa funkcionális megfelelőjének tekinthetők

8. Fetchapi.html

A fetchapi.html menüpont már egy jóval összetettebb feladatrészt tartalmazott, mert itt szerveroldali adatbázis-tárolásra is szükség volt. A fejlesztés során ehhez SQL alapú adatbázist kellett létrehoznunk. Mivel a munkát Visual Studio Code környezetben végeztük, telepítettünk egy SQLite bővítményt, valamint hozzá az SQLite Viewer kiegészítőt.



8. ábra VS Code bővítmények

Ez első ránézésre praktikus megoldásnak tűnt, mert a fejlesztési környezetben belül tudtuk kezelni az adatbázist. A gyakorlatban azonban hamar kiderült, hogy ez a megoldás korlátozott. Különösen hiányzott belőle egy egyszerű exportálási lehetőség, amely megkönnyítette volna az adatok átvitelét. Emiatt egy online konverterhez fordultunk, amely a txt formátumban kapott adatokat .sql kiterjesztésű, beimportálható fájlalká alakította. Ez a lépés ugyan segített, de nem volt tökéletes: bizonyos adattípusokat hibásan ismert fel a rendszer. Például az Id mezőt Double típusként próbálta létrehozni, miközben nekünk Integer típusra volt szükségünk automatikus számozással. Ezt ezért manuálisan javítottuk, hogy a rekordok azonosítása egyszerű és ütközésmentes maradjon. Az adatbázist végül kézzel hoztuk létre a VS Code környezetében. A backend kialakításánál a tanórai PHP kód szolgált kiindulási alapként, de azt rögtön láttuk, hogy az adatbázis-kapcsolatot kezelő rész nem használható változtatás nélkül. Más megoldás szükséges ugyanis XAMPP és MySQL esetén, mint SQLite használatakor. Emiatt a db.php fájlt saját környezetünkhöz kellett igazítani. Eleinte több kiolvasási probléma is jelentkezett, ezért rövidebb és egyszerűbb kódokkal próbáltuk meghatározni a hiba forrását. Csak akkor tértünk vissza a tanórai, strukturáltabb megoldáshoz, amikor az alapkapsolat már stabilan működött. Ekkor alakítottuk át a read.php fájlt a saját adatbázisunknak megfelelően, ezt úgy oldottuk meg, hogy a read.php fájlhoz az alábbi módosításokat adtuk hozzá, hogy engedélyezzük a hozzáférést:

- `header("Content-Type: application/json");`
- `header("Access-Control-Allow-Origin: *");`
- `header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS");`
- `header("Access-Control-Allow-Headers: Content-Type");`

Hasonló logikával építettük fel a kliensoldali JavaScript részt is a `fetchapi.js` fájlban. A HTML oldalt a már meglévő, közös stílusok felhasználásával formáztuk, így a Fetch API menüpont is vizuálisan egységes maradt. Ez a weboldalrész a `localhost:8000`-es porton futott, ami később az Axios menünél is fontos szerepet kapott.

9. Axios.html

Az `axios.html` menüpontnál abból az előnyből indulhattunk ki, hogy az adatbázis-kezelés alapjai ekkorra már nagyrészt elkészültek. Ugyanakkor ez a rész még így is számos új

kihívást hozott, mivel itt React környezetben kellett Axios segítségével megvalósítani a szerveroldali adatkezelést. A fejlesztés első technikai lépése az Axios telepítése volt, amelyet a VS Code termináljában az `npm install axios` paranccsal végeztünk el. Az alkalmazás megírásánál ismét a tanórai mintakódot vettük alapul, de ezt jelentősen testre kellett szabni. A legnagyobb kihívást az jelentette, hogy a backend és a frontend külön porton futott. A saját fejlesztői gépen a localhost:8000 porton működött a PHP-s, adatbázisra épülő rész, míg a Reactet futtató npm fejlesztői kiszolgáló a localhost:5173 portot használta. Kezdetben nem fordítottunk erre elegendő figyelmet, ezért nehezen volt átlátható, miért nem kommunikál megfelelően egymással a két rendszer. Ebből adódóan előfordult, hogy vagy az adatbázis elérése nem működött, vagy a React alkalmazás nem reagált a várt módon. A probléma megoldásához a `read.php` fájlba CORS-engedélyező fejlécsorokat illesztettünk be. Ennek részeként beállítottuk a Content-Type értéket, engedélyeztük a külső eredetű hozzáférést, valamint megadtuk az engedélyezett metódusokat és fejlécmezőket is. Ezzel a szerver képes lett fogadni a különböző portra érkező kéréseket. Emellett az `axios.jsx` fájlban is pontosítani kellett az API elérési útját, vagyis rögzítettük, hogy a `read.php` a localhost:8000 címen érhető el. Miután ez a kapcsolat stabilan működött, a felhasználói élmény javításával is foglalkoztunk. Itt merült fel az az ötlet, hogy a rendszer ne pusztán egy szöveges kiírással jelezze a sikeres műveleteket, hanem a böngésző jelenítsen meg értesítést. Ez azért bizonyult előnyösnek, mert így a visszajelzés nem zavarta meg a felület vizuális szerkezetét. Emellett a `read.php` állapotüzeneteit magyar nyelvűre módosítottuk, vagyis például az angol sikerüzenetek helyett „Sikeres létrehozás!” formájú visszajelzéseket adtunk. Az Axios oldal így nemcsak technikailag valósította meg a beadandó előírásait, hanem használhatóság és megjelenés szempontjából is egységesen illeszkedett a teljes alkalmazásba.

10.Oojs.html

A projekt következő lépéseként az OOJS (Object-Oriented JavaScript) menüpont kialakítása történt meg. Ennél a feladatrésznél a beadandó előírása szerint egy grafikus, objektumorientált szemléletben megvalósított JavaScript alkalmazást kellett készítenünk.

A fejlesztés kezdeti szakaszában először egy webshop jellegű megoldásban gondolkodtunk, amely a cukrászda termékeit jelenítette volna meg. Ez az elképzelés azonban nem bizonyult megfelelőnek, mivel a feladatleírás kifejezetten grafikus, interaktív alkalmazás készítését várta el, nem pedig egy adatlistázó vagy webshop jellegű felületet. Emiatt újragondoltuk a koncepciót

és mivel a teljes projekt egy cukrászda témára épült, végül innen merítettünk ötletet, és így jutottunk el a szerencsesüti alkalmazás gondolatához. Ez jól illeszkedett a tematikához, ugyanakkor lehetőséget adott látványos animációk és objektumorientált működés megvalósítására is.

A tervezési szakaszban először egy egyszerű vizuális elképzelést alakítottunk ki. Ennek lényege az volt, hogy a képernyő közepén két darab sütemény jelenjen meg, amelyek enyhén „rezgő” animációval hívják fel magukra a figyelmet. A felhasználó ezekre kattintva indíthatja el az alkalmazás működését, amely során a süti „kinyílik”, és megjelenik egy üzenet. A kezdeti megvalósítás során először egy egyszerű, alap kinézetet készítettünk el. Ebben a szakaszban a hangsúly nem a megjelenésen, hanem a működésen volt. A süti objektumorientált módon került kialakításra, és sikerült elérni, hogy kattintásra animáció induljon, majd megjelenjen a jóslat szövege.



9. ábra Szerencse süti kezdeti megvalósítása

Amikor az alkalmazás működése már megfelelő volt, több megjelenítési problémát is észleltünk. Ekkor még teljesen el volt csúszva az oldal kinézete, nem illeszkedett a többi oldalhoz, és a teljes oldal egy egyszerű fehér háttérrel jelent meg. Ebben az állapotban az oldal ugyan funkcionálisan működött, de vizuálisan nem illett bele az alkalmazás egységes arculatába. Ezután kezdtük el a kinézet fokozatos finomítását, azaz kialakítottuk a teljes oldal elrendezését, amelyben a fejléc és a lábléc közé került a központi tartalom. A végleges megjelenés kialakításánál külön figyelmet fordítottunk arra, hogy az OOJS oldal is illeszkedjen a már kialakított cukrászda arculathoz. Ennek érdekében ugyanazt a színvilágot használtuk, mint a többi oldalon, vagyis a sötétbarna és a világos bézs árnyalatokat. A középső tartalom egy lekerekített sarkú, árnyékolt „kártya” formájában jelenik meg, amely kiemeli a szerencsesüti animációt.



10. ábra Oojs.html kezdeti kinézete

Az animáció során a süti két félből áll, amelyek kattintás hatására szétválnak, majd megjelenik a jóslatot tartalmazó „papír”. Ezt követően egy gomb segítségével új süti kérhető,

így az alkalmazás többször is használható. Az oldal alján elhelyezett információs elemek („OOJS alapú vezérlés”, „CSS animációk”, „Reszponzív megjelenés”) azt a célt szolgálják, hogy röviden összefoglalják, milyen technológiák és megoldások kerültek alkalmazásra ezen az oldalon.



11. ábra Végleges oojs.html kinézet

A fentiekén túl fontos még lezárásként kiemelni, hogy az OOJS alkalmazásban az alábbi objektumorientált elemeket alkalmaztuk, amelyek a feladat leírása szerint elvárásként szerepeltek:

- class
- constructor
- metódusok
- extends és super
- document.body.appendChild
-

11.A Nettarhely.hu-ra való feltöltés

A segéddokumentumokban leírt módon létrehoztuk a nettarhely.hu oldalon az ingyenes tárhelyet. A tárhelyet FTP kapcsolaton keresztül értük el, melyhez Total Commandert használtunk. Létrehoztuk az „eviesdavid” elnevezésű MySQL adatbázist és importáltuk benne az adatokat. A feltöltés során a db.php fájlt módosítanunk kellett, hogy megtalálja az adott adatbázist. Ez (számunkra is meglepő módon) elsőre működött. Sajnos a feltöltés után nem működtek a React alapú oldalak, mire rájöttünk, hogy a „build” részt elfelejtettük. Ehhez frissíteni kellett a vite.config.js fájlt, hogy a React alapú html-eket elérje. Ezután

parancssorosan lefuttattuk az `-npm run build-` parancsot, és a `dist` mappában létrehozott állományokat felmásoltuk a tárhelyre, és így már probléma nélkül működött az oldal React-os része is. A PHP verziókat a 8.2-t állítottuk be, mivel talán ez egy biztonságosabb verzió a régiekhez képest, és a kódunk kompatibilis ezzel a verzióval.

Az elérés:

Email: info.alfadelta@gmail.com

Jelszó: eviesdavidwebprog18

12.Rövid használati útmutató a honlaphoz

A weboldal használata tudatosan egyszerű és áttekinthető módon lett kialakítva. A felhasználó a főoldalra érkezve először röviden megismerheti az Édes Liget cukrászda bemutatkozását, majd a felső menüsor segítségével érheti el az egyes funkcionális aloldalakat.

A JavaScript menüpont alatt egy tömbben kezelt CRUD alkalmazás található, ahol új süteményadatok vehetők fel, a meglévők módosíthatók, illetve törölhetők.

A React menüpont hasonló funkciót kínál, mint a Javascript CRUD alkalmazása, azonban itt a műveletek már React alapú megoldásban valósulnak meg.

Tovább lépve az SPA menüpont két, gombok segítségével váltható React alkalmazást tartalmaz a digitális óra és a nap számláló applikáció, amelyek ugyanazon az oldalon belül jelennek meg.

A Fetch API és az Axios oldalak már szerveroldali adattárolással működnek, ezért ezek a menüpontok a háttérben adatbázissal is kommunikálnak.

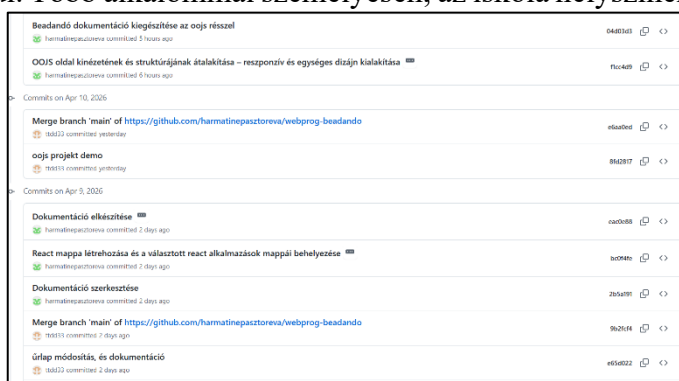
A menüsor minden oldalon elérhető, kisebb képernyőn pedig hamburger menü formájában jelenik meg. Így a honlap számítógépen és kisebb kijelzőn is könnyen használható marad.

Az OOJS menüpont alatt egy objektumorientált JavaScript alapú grafikus alkalmazás található, amely egy szerencsesüti működését jeleníti meg. A felhasználó a süteményre kattintva megnyithatja azt, ekkor egy véletlenszerű üzenet jelenik meg. Az alkalmazás ezt követően egy gomb segítségével újraindítható, így új szerencsesüti és új üzenet kérhető.

13.Összegzés

A projekt megvalósítása során a feladatokat végig közösen, szoros együttműködésben végeztük el. Bár a GitHub repository-ban a commitok alapján időnként úgy tűnhet, hogy az egyik fél aktívabb volt, a valóságban a fejlesztési folyamat minden lépését együtt beszéltük át. Folyamatos online kapcsolatban álltunk egymással, így a legtöbb esetben a commitolás előtt már mindketten átnéztük és egyeztettük a módosításokat.

Az összes HTML oldal (index, javascript, react, spa, fetchapi, axios, oajs) kialakítása közös munkával történt, a különbség inkább abban jelent meg, hogy éppen ki végezte el a commitolást a GitHub Desktop alkalmazáson keresztül. Több alkalommal személyesen, az iskola helyszínén is együtt dolgoztunk, ilyenkor jellemzően az egyik csoporttag számítógépén folyt a munka. Ez magyarázza, hogy egyes napokon az egyik félhez köthető több commit jelenik meg a verziókövetésben. A GitHub repository-ban jól látható a projekt folyamatos fejlesztése.



12. ábra Commit lista egy része

A commitok száma és időbeli eloszlása igazolja, hogy a feladat több lépésben készült el. A contributors (harmatinepasztoreva – Harmatiné Pásztor Éva (JBDAQO), ttd33 – Deák Dávid (OPFHRV)) részben mindkét csoporttag megjelenik, ami alátámasztja a közös munkavégzést.

A projekt során a GitHub Desktop használata mindkettőnk számára új tapasztalat volt, és pozitív meglepetésként ért minket, hogy mennyire hatékonyan támogatja a közös munkavégzést. Ennek köszönhetően nemcsak a beadandó feladatot tudtuk strukturáltan és átlátható módon megvalósítani, hanem jelentős tapasztalatot is szereztünk a verziókövetés és a csapatmunka gyakorlati alkalmazásában. Összességében a projekt egyik legnagyobb pozitív élménye éppen az volt, hogy megtanultuk, hogyan lehet egy közös fejlesztést hatékonyan megszervezni és lebonyolítani GitHub környezetben.

14. Források

A projekt elkészítése során a következő forrásokat és segédanyagokat használtuk fel:

- Web-programozás-1-Előadás című tananyag PDF
- A tanár úr által biztosított megoldások.zip fájl
- A megadott adatbázis-forrásokat tartalmazó Google Drive mappa: „1-14-Cukrászda-3 táblás”
- GitHub repository: <https://github.com/harmatinepasztoreva/webprog-beadando>
- React mintaprojektek gyűjteménye: <https://github.com/ianshulx/React-projects-for-beginners>
- Canva a logó elkészítéséhez - <https://www.canva.com/>
- Coolers a színpaletta kiválasztásához - <https://coolers.co/>
- Aspose online txt-sql konverter az adatbázis konvertálásához- <https://products.aspose.app/cells/conversion/txt-to-sql>